Hardware Implementation of Neural Network Inference Using SystemVerilog - Phase 1: MAC Arithmetic and Activation Blocks

1. Introduction

The main objective of this phase is to demonstrate a basic understanding of Register Transfer Level (RTL) by designing and implementing a Multiply-Accumulate (MAC) unit and a Rectified Linear Unit (ReLU) in SystemVerilog and using Testbenches to verify them.
These components will later be put together to form the computational foundation of a simulated neural network with a structure of 2 → 4 → 2.

This phase consists of four main sections, three of which are designed in SystemVerilog & GTKWave.

Section one will provide a concise description of a neural network's general formula.

Section two will focus on implementing a Multiply-Accumulator (MAC) unit.

Section three involves designing and implementing a Rectified Linear Unit (ReLU) in SystemVerilog.

Section four will be derived from the two above sections and put everything implemented so far together, and verify the components' behavior using Testbenches and Design Under Test (DUT).

2. Neural Network

A neural network is mathematically defined as a layer of **neurons**, each being a composition of a non-linear function. Each neuron gets an input, multiplies it by its weight term, adds a bias, and passes the result to a non-linear function. Note that this section will cover the general math representation of a neural network, not providing a deep learning lecture. The main purpose is to provide readers with a good understanding of the relationship between the components being implemented in SystemVerilog and later used in a Neural Network.

A neural network has a general formula as shown below:

$$y = f(w_i x_i + w_{i+1} x_{i+1} + \ldots + b) = f(\Sigma w_i x_i + b)$$

Where:

- $i \in \{1, 2, 3, \ldots, n\}$
- x_i : input features
- w_i : learned weights
- b : bias
- $f(.)$: activation function

The key point here is the sigma notation and the term being computed. Note that *w* would get **multiplied** by *x* in each **cycle** and be **accumulated** with the previous value.

3. Multiply-Accumulate (MAC)

In the previous introductory section, the formula of a Neural Network's neuron and the notations have been introduced. In this section, the motivation behind using a MAC in a neuron is going to be discussed.

The **multiply–accumulate (MAC)** operation is a common step that computes the *product* of two numbers and adds that product to an *accumulator*. The hardware unit that performs the operation is known as a **multiplier–accumulator (MAC unit)**; the operation itself is also often called a MAC operation.
In neural networks, a MAC generally has the form: $acc = acc + w * x$

In hardware, a MAC is not a "one operation". It is a data pipeline block made of:

- **Multiplier**: $w * x$, which produces a product
- **Adder**: $acc + product$
- **Register (memory)**: Stores $acc(next) \leftarrow acc(current\ result)$

Hardware replaces many multiplications and additions with a reusable engine known as the MAC, which is a state machine with a register.

To better demonstrate how MAC functions, other hardware terminology and concepts should be explained:

3.1 Combinational Circuit

There are mainly two types of circuits that are going to be discussed in this phase of the project. The first one is the combinational circuit. Combinational Circuits are those whose output depends only and immediately on their inputs. They have no register, i.e., dependence on past values of their inputs. Sequential Circuits, on the other hand, are components that can store information and have a register. [1]

In the MAC equation above, notice the term Multiplier. This term was computed using a combinational circuit. The produced value of the multiplication is immediate and dependent on only its inputs, in this example, *w & x.*
Moving on to the next part of the equation, there is an *acc* term, which was added to the **previous value of acc**. The key point here is the fact that there should be a **register** to keep track of the previous values of acc. This can't be done using combinational circuits. Here's where Sequential Circuits would be useful.

3.2 Sequential Circuit

Sequential circuits act as storage elements and have a register. They can store, retain, and then retrieve information when needed at a later time. A sequential circuit is specified by a time sequence of inputs, outputs, and internal states. A block diagram of a sequential circuit is shown in Fig. 1. [1]

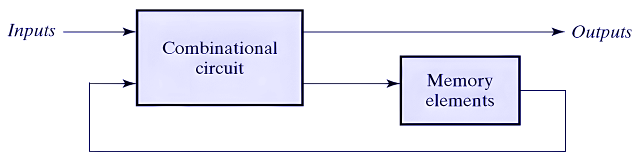

Figure 1. Block diagram of a sequential circuit [1, p. 191]

3.3 Synchronous Sequential Circuit

Synchronous sequential circuits are defined by their signals at discrete times and update their state according to a clock signal, often labeled as *clock* or *clk*. A clock generator provides a periodic train of clock pulses, which are distributed throughout the system, ensuring that storage elements are updated with each pulse. These circuits are called *clocked sequential circuits* because their activity is synchronized with the clock pulses.
Adapted from [1, p.191]

3.4 Flip-Flops

The storage elements (memory) used in clocked sequential circuits are called *flip-flops.* A flip-flop is a binary storage device capable of storing one bit of information. In a stable state, the output of a flip-flop is either 0 or 1. The block diagram of a synchronous clocked sequential circuit is shown in Fig. 2. [1, p.192]

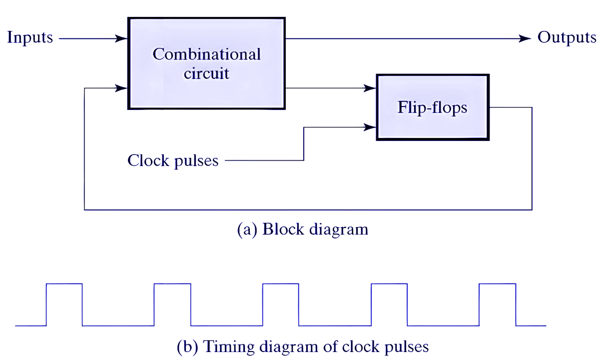

Figure 2. Synchronous clocked sequential circuit [1, p. 192]

Flip-flops are one of the main building blocks of synchronous circuits, and outputs are formed by a combinational logic function of the inputs to the circuit or the values stored in the flip-flops, or both, which is the case of this work.

The new value is stored (i.e., the flip-flop is updated) when a pulse of the clock signal occurs, changing state only at these predetermined intervals. *Adapted from* [1, p.192]

A momentary change in the control input, called a *trigger*, causes the flip-flop to switch states. A clock pulse transitions from 0 to 1 and back from 1 to 0.

As shown in Fig. 3, the positive transition is defined as the positive edge and the negative transition as the negative edge:

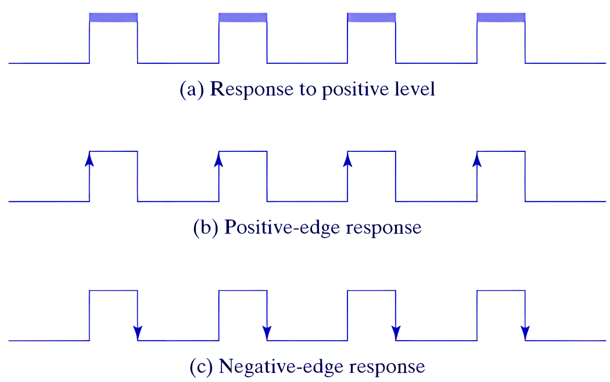

Figure 3. Clock response in flip-flop [1, p.197]

3.5 MAC SystemVerilog Implementation

This section begins with code snippets, which together form the SystemVerilog implementation of the MAC unit. The code consists of several terms and notations that are explained as follows.

3.5.1 MAC Module

Each of the components in this work was written in a separate `.sv` file. This has been done to keep the project modularized. In SystemVerilog, each unit begins with the keyword `` `module` `` followed by the name of the block, in this case, `mac_seq`. The module processes input and output signals. These signals store data in **2's complement** format and can represent signed or unsigned data depending on the problem being addressed.

```
module moduleName()
```

3.5.2 Signal Width and Signedness

The signals that were used in the MAC module can be summarized and described in the following table:

Signal	**Type**	**Width**	**Description**
clk	logic	1 bit	System clock
reset	logic	1 bit	Synchronous reset
X	signed	4 bits	Input value
w	signed	4 bits	Weight
acc	signed	8 bits	Accumulator

Table 1. A description of MAC module signals

The type and signedness of each signal are explained below:

- **Clock:** The clock (or clk for short) is a periodic digital signal that transitions between logic 0 and logic 1. It is used to synchronize a synchronous circuit and trigger updates of sequential elements on its active edge.
- **Reset:** The reset is a digital signal that is used to initialize the register and force the accumulator (acc) to 0.
- **X:** The input value *X* is a signed 4-bit vector (bus) that stores 2's complement values in 4 bits.
- **w:** W, similar to X, is a signed 4-bit vector (bus) that stores 2's complement values in 4 bits.
- **Acc:** The resulting multiplication of w and X is stored in an 8-bit vector called acc.

A few points worth mentioning. By default, each signal with the type **logic** in SystemVerilog is **unsigned**. For instance, the following signals are **unsigned**, **one-bit** signals with two states, either 0 or 1:

```
input logic clk
input logic reset
```

Note that, as mentioned in the above table, `X` and `w` are the two **signed vector** inputs. In SystemVerilog, to declare a **signed vector** signal, the keyword `signed`, along with the **packed dimension [msb:lsb]** and the signal name, should be used:

```
input/output logic signed [msb:lsb] signalName
```

By the use of square brackets, a **vector** or a **bus** will be created. The signal width is specified using the range [msb:lsb]. This is especially crucial for storing neurons' weights/biases, which are represented in multiple bits and can store positive or negative values.

```
input logic signed [3:0] X
input logic signed [3:0] w
output logic signed [7:0] acc
```

For simplicity, the resulting value of the multiplication was declared as a 2N-bit-width signal. Meaning, if each of the inputs has N bits, the resulting bits of the multiplication should be stored in 2N bits. In this case, X and w each have 4 bits, and the result, acc, would have 8 bits:

$$N - bit\ signed \times N - bit\ signed \rightarrow use\ about\ 2N\ bits$$

3.5.3 Overflow

One important setback of using just N bits (input signal's width) as the resulting signal width is **overflow**. This could be the result of a computation (in this work, addition or multiplication), where the output number can't be stored in the initialized signal, and the resulting width is greater than the declared width. In simple words, overflow happens when the maximum limit of a particular signal is exceeded.
To overcome this issue, a signal width of 2N, i.e., 8 bits, is initialized.

This topic will be covered in depth in Phase 3, where the objective will be designing a hardware implementation simulation of the trained neural network from Phase 2.

3.5.4 Combinational Product

MAC consists of two main circuits. The multiplication part is computed from a combinational circuit, in which w and X are being multiplied immediately and are not dependent on the previous inputs.
As shown below, the resulting multiplication was stored in an 8-bit signal called `product`. Note that declaring the resulting signals with explicit width before any computations is often considered a good practice.
The keyword `` `assign` `` was used to compute the product of w and X. Using this keyword, the multiplication was being computed continuously by **combinational** logic.

```
logic signed [7:0] product;
assign product = X * w;
```

3.5.5 Sequential MAC

The sequential part of the MAC is implemented in an `always_ff` procedural block and is used to model synthesizable sequential logic, such as flip-flops.
In SystemVerilog, the `@` symbol is the event control operator used to delay execution or trigger a process based on specific signal changes or events. In this work:

```
always_ff @(posedge clk)
```

triggers when the clock transitions from 0 to 1 (rising edge).

In the above `always_ff` block, the computations are being processed only when a `posedge` happens. This means that *acc* was updated only when there was a positive rising edge clock. If the clock was transitioning from 1 to 0, the update code snippet inside the `always_ff` block would not occur, and the values stayed constant between the clocks.

MAC updates the accumulator once per positive clock edge. For N inputs, N clocks will be computed. This is called **execution cycles**.

The next signal that controls the computation and is going to be introduced is the `` `synchronous reset` ``. It is called synchronous because it is used inside the synchronous sequential block `always_ff`.

As shown in the following code, if reset is in state 1, *acc* would be initialized to 0. The main purpose of reset is to set the accumulator (*acc*) to 0 and avoid any random initialization.
If reset is in state 0, no initialization is done, and therefore, the value of *acc* will be updated.
Note that the value of acc was being computed while keeping track of the previous values of acc.

```
if (reset)
    acc <= 0;
else
    acc <= acc + product;
```

The key point here is the symbol <=, which is a non-blocking assignment operator. It is used inside sequential blocks, such as `always_ff` in this work, to model **flip-flops** and **registers**. With non-blocking assignments, the execution of the code continues before the assignment happens. Meaning all RHS expressions are evaluated first (using the old values from before the clock edge), then all assignments occur simultaneously after the evaluation. This makes it suitable for modeling synchronous (clocked) logic where operations need to happen in parallel.

A comparison between `` `=` `` used in `product`, and `` `<=` `` used inside the if-block is shown in the following table to demonstrate a better understanding:

Feature	<= (non-blocking)	= (blocking)
Order	All RHS are evaluated in parallel before any LHS changes	Executed sequentially in the order written
Behavior	Simulates edge-triggered registers	Simulates combinational logic
Usage	Sequential logic (flip-flops, registers)	Combinational logic (`always_comb`) or testbench initial blocks

Table 2. Comparison between = and <= operators in SystemVerilog

4.1 Rectified Linear Unit (ReLU)

Unlike the previous sections, where the main focus was on sequential computations and synchronous circuits, this section will go through the details of implementing an activation function called **ReLU** using **combinational** logic. **ReLU** is considered an essential part of a neuron's output, by introducing non-linearity and complex pattern recognition abilities. Mathematically, it is defined as:

$$ReLU(x) = max(0, x)$$

OR

$$ReLU(x) = \{$$

$$if\ x >= 0;\ x$$

$$if\ x < 0;\ 0$$

$$\}$$

Graph:

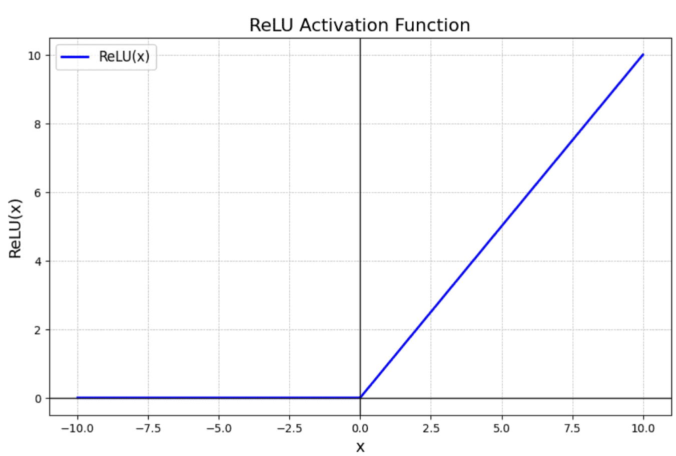

Figure 4. ReLU - Image from GeeksforGeeks

4.2 ReLU Implementation in SystemVerilog

This section will focus on the implementation details of ReLU in SystemVerilog.

In the previous section, the output of the MAC was a vector and had a width of 8. That output will be used as the input signal of the ReLU. Note that ReLU only does a filtering on the value. It doesn't change the width of the input signal. Therefore, the resulting output signal of ReLU will have 8 bits as well.

```
input logic signed [7:0] in,

output logic signed [7:0] out
```

ReLU was computed using two versions. The first approach was using sign-bit inspection and a combinational circuit. The second version followed the approach of the **Ternary Operator**, which is a much simpler method. Since the output of ReLU depends only and immediately on its input, both of the approaches are considered **combinational**.

The first version was computed using the following procedural block:

```
always_comb begin : ReLU
```

The `always_comb` procedural block is used to describe combinational logic. The output is automatically recomputed whenever one of the inputs changes.

In this approach, ReLU works by sign-bit inspection. Since the signal uses two's complement signed representation, an MSB of 1 indicates a negative value; thus, the output will become 0. Otherwise, if the MSB is 0, the number is non-negative (zero or positive), and the output will be equal to the input.

In code:

```
if (in[7] == 1)

    out = 0;

else

    out = in;
```

Alternatively, since the deal here is with **combinational circuits**, another version of ReLU was written using the Ternary Operator as follows:

```
assign out = (in < 0) ? 0 : in;
```

This version could be written without the need for an always block. This version is indeed much cleaner, since it does not deal with the MSB and indexing, and is considered a good design when working with large 2's complement numbers.

5.1 Verification Methodology

The main focus of this section is on Testbenches and verification for the units built so far, i.e., MAC and ReLU.

In SystemVerilog, a Testbench tests a Design Under Test (DUT), which is the hardware design (e.g., an RTL module) being verified. The testbench is used for functional verification to find design bugs.

5.2 MAC Module Verification

This section focuses on testbenches and writing a DUT for the module MAC. Therefore, the input and output signals of the module MAC should be first declared.

```
logic clk;

logic reset;

logic signed [3:0] X;

logic signed [3:0] w;

logic signed [7:0] acc;
```

To design a DUT for a module, the instantiated module was named `dut`, which is a common naming convention in verification environments. The declared signals needed to match the module's signals in the order:

```
mac_seq dut(

    .clk(clk),

    .reset(reset),

    .X(X),

    .w(w),

    .acc(acc)
```

```
);
```

A clock with a 10 ns period was generated by inverting the clock signal every 5 ns using the following code:

```
always #5 clk = ~clk; // toggle clock every 5 ns
```

The testbench consists of the following components:

- `$dumpfile` task, which is used to specify the name of the VCD file
- `$dumpvars` task, which is used to specify which variables should be recorded in the file indicated by `$dumpfile`
- `$monitor`, which helps to automatically print out variable or expression values whenever the variable or expression is in its argument

```
initial begin

    $dumpfile("mac.vcd");
    $dumpvars(0, mac_seq_tb);

    $monitor("t=%0t | clk=%0d reset=%0d X=%0d w=%0d acc=%0d",
                 $time, clk, reset, X, w, acc);
```

The declared signals were initialized as follows. Note that the values have been assigned randomly for learning purposes:

```
    // initialize signals
    clk = 0;
    reset = 1;
    X = 0;
    w = 0;
```

A delay of 10 ns was introduced before applying the input stimulus.

```
#10; // time delay
```

By the time t=10 ns, the signal values have been assigned new values:

```
// update signals
reset = 0;
X = 2;
```

```
w = 3;
```

After the initialization, another time delay of 50 ns has been set, and the performance has been monitored for 50 ns:

```
#50;
```

5.2.2 MAC Waveform Analysis

A table of the MAC waveform is provided below:

Time	clk	reset	X	w	acc (Hexa)
5	↑	1	0	0	00
15	↑	0	2	3	06
25	↑	0	2	3	0C
35	↑	0	2	3	12
45	↑	0	2	3	18
55	↑	0	2	3	1E

Table 3. Accumulator values at positive clock edges

Sample explanation of a positive clock edge: At 15 ns, the MAC computes $0+(2\times3)=6$.
The resulting value is stored in the accumulator register.

Notably, the table above demonstrates the positive edge clocks, i.e., the clock on which the *acc* value was updated.

The GTKWave images have been provided in the following. Notably, the values of *acc* are written in **Hexadecimal**.

t = 0 ns to t = 55 ns

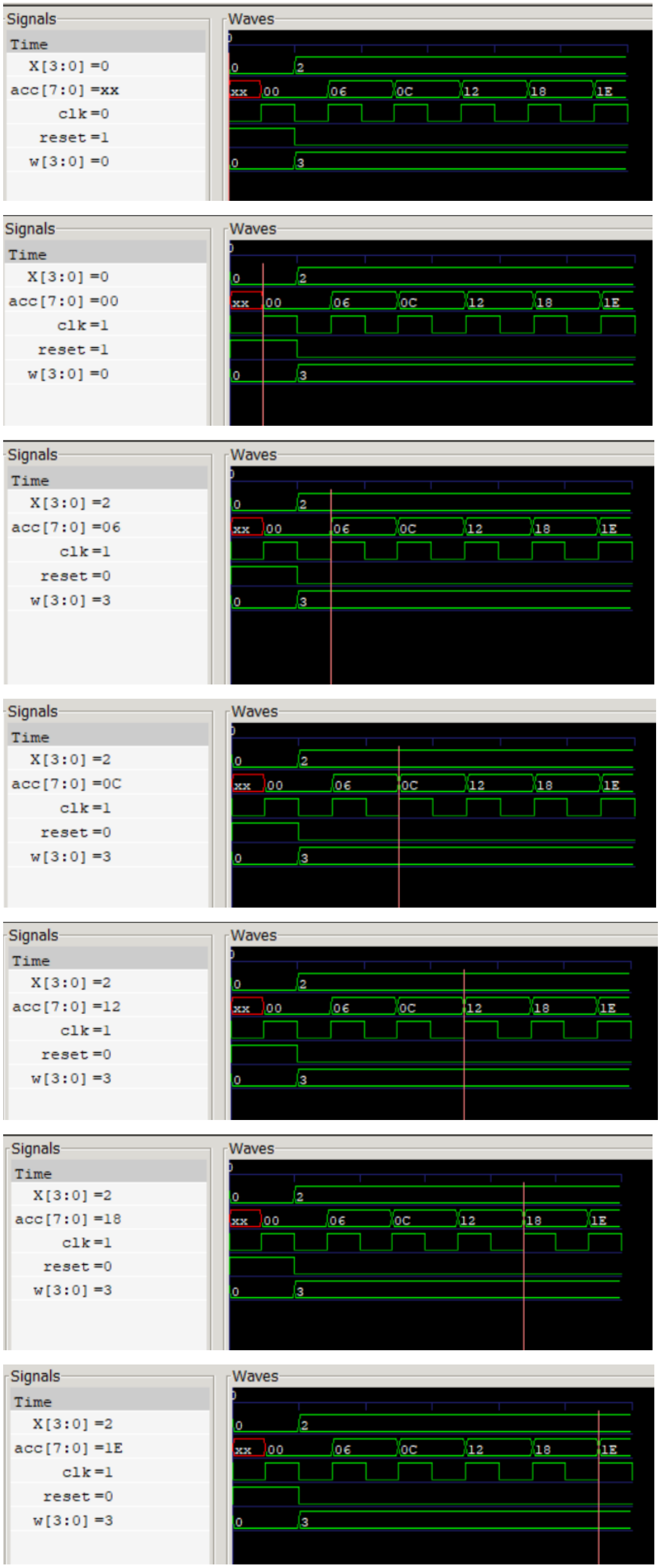

Figure 5. MAC waveform during accumulation

5.3 ReLU Module Verification

A few testbenches have been provided to verify the ReLU behavior as follows:

The signal widths are the same as the ReLU module:

```
logic signed [7:0] in;

logic signed [7:0] out;
```

For this DUT, the `$display` task and a time delay of 10 ns have been used:

```
relu dut(

    .in(in),

    .out(out)
```

```
in = -6;

#10;

$display("%0d | %0d", in, out);


in = -1;

#10;

$display("%0d | %0d", in, out);


in = 0;

#10;

$display("%0d | %0d", in, out);


in = 5;

#10;

$display("%0d | %0d", in, out);


in = 12;

#10;
```

```
$display("%0d | %0d", in, out);
```

A summarization of the output can be found in the next table as well:

input	**output**
-6	0
-1	0
0	0
5	5
12	12

Table 4. ReLU module verification

Summary

In this work, the hardware implementation of neural network inference in SystemVerilog has been described, with a focus on the design of a Multiply-Accumulate (MAC) unit and a Rectified Linear Unit (ReLU) verified through testbenches. The mathematical framework of the neural network has been explained, highlighting the role of the MAC in processing inputs through weighted multiplication and accumulation. A distinction has been made between combinational circuits, which depend only on current inputs, and sequential circuits, which store previous values using registers.

References

[1] M. M. Mano and M. D. Ciletti, Digital Design with an Introduction to the Verilog HDL, 5th ed. New York, NY, USA: Pearson, 2013.

By Kiana Jafari - Computer Engineering Student